

Student Record Management System

1. Project Overview

The Student Record Management System is a console-based Python application that allows users to add, store, and display student records using text files. The application demonstrates Python file handling, exception handling, input validation, and menu-driven programming through a practical real-world example.

2. Problem Statement

Managing student records manually can become difficult as the number of records increases. This project provides a simple file-based solution for storing, retrieving, and displaying student information while ensuring proper validation and exception handling using Python.

3. Features

Student Management

- Add student records
- Store records in a text file
- Display student records

Input Validation

- Validate student name
- Validate age
- Validate GPA

Exception Handling

- File Not Found
- Empty File
- Invalid Data Format
- IO Error

User Interface

- Menu-driven application

- Formatted output

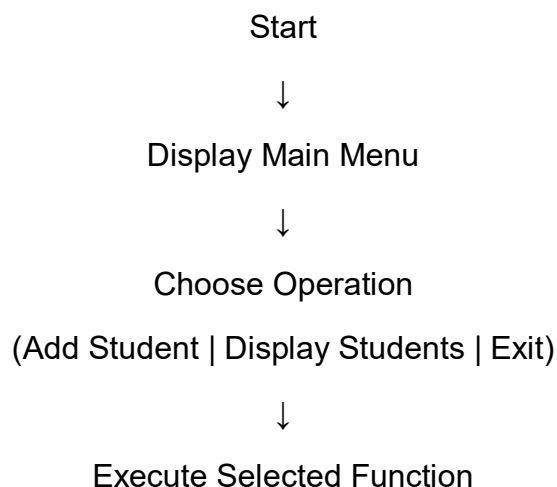
4. Technologies Used

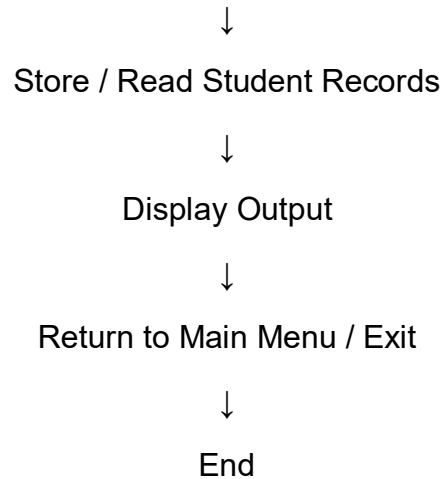
- Python 3
- Visual Studio Code
- Text File (students.txt)

5. Python Concepts Used

- Functions
- File Handling
- Read Mode (r)
- Append Mode (a)
- with Statement
- Exception Handling
- FileNotFoundError
- IO Error
- Value Error
- Lists
- Loops
- Conditional Statements
- String Methods
- Formatted Output (f-strings)

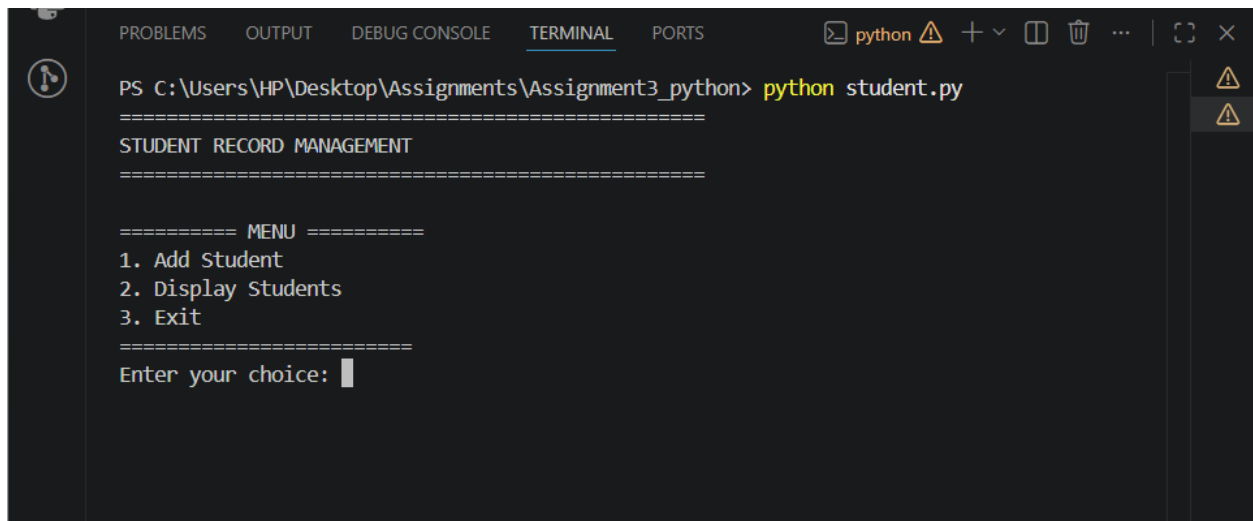
6. Program Workflow





7. Output Screenshots and Explanation

Screenshot 1: Main Menu



```
PS C:\Users\HP\Desktop\Assignments\Assignment3_python> python student.py
=====
STUDENT RECORD MANAGEMENT
=====

===== MENU =====
1. Add Student
2. Display Students
3. Exit
=====
Enter your choice: █
```

Explanation:

The application starts with a menu-driven interface that provides options to add student records, display stored records, or exit the program. The menu validates user input before performing the selected operation.

Screenshot 2: Adding Student Record

```
===== MENU =====
1. Add Student
2. Display Students
3. Exit
=====
Enter your choice: 1
Enter the student's name:john
Enter student's age:22
Enter a student GPA value:2
Student details saved successfully!

===== MENU =====
1. Add Student
2. Display Students
3. Exit
=====
Enter your choice: |
```

Explanation:

The user enters the student's name, age, and GPA. The application validates each input and appends the student details to the students.txt file without overwriting existing records.

Screenshot 3: Display Student Records

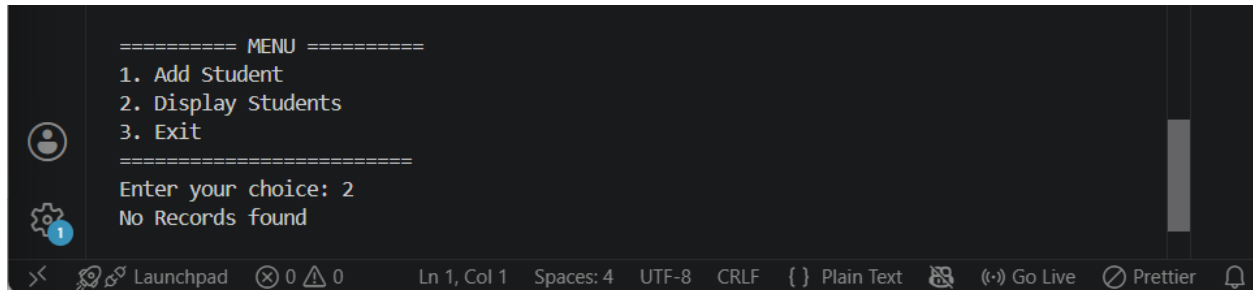
```
===== MENU =====
1. Add Student
2. Display Students
3. Exit
=====
Enter your choice: 2

Student Records
-----
Name      : john
Age       : 22
GPA       : 2.00
-----
Name      : rahul
Age       : 19
GPA       : 3.56
-----
Name      : priya
Age       : 22
GPA       : 3.92
-----
```

Explanation:

The application reads all records from the text file, validates each record before displaying the student information, and displays each student's information in a clean and organized format.

Screenshot 4: Exception Handling

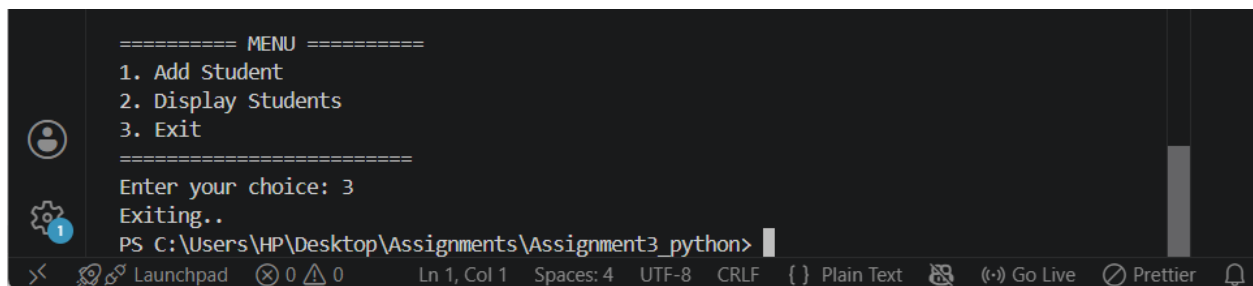


```
===== MENU =====
1. Add Student
2. Display Students
3. Exit
=====
Enter your choice: 2
No Records found
```

Explanation:

The program handles exceptions gracefully by displaying appropriate error messages for situations such as missing files, empty files, or incorrectly formatted data without terminating unexpectedly.

Screenshot 5: Exit Program



```
===== MENU =====
1. Add Student
2. Display Students
3. Exit
=====
Enter your choice: 3
Exiting..
PS C:\Users\HP\Desktop\Assignments\Assignment3_python>
```

Explanation:

The user selects the Exit option to terminate the application. The program closes safely after displaying a confirmation message.

8. Learning Outcomes

- Read and write data using text files
- Implement menu-driven applications

- Validate user input
- Handle file-related exceptions
- Organize programs using functions
- Apply Python file handling concepts to a real-world application
- Improve problem-solving and debugging skills

9. Conclusion

The Student Record Management System was successfully developed using Python file handling, exception handling, functions, loops, conditional statements, and formatted output. The application efficiently stores and retrieves student records while ensuring proper validation and error handling, providing a reliable and user-friendly console-based solution.